

Progetto TPSIT

Una Web App che permetta di prenotare vari punti di interesse, appartenenti ad una determinata categoria (in questo caso hotel).

Software utilizzati nello sviluppo:

- IntelliJ IDEA (IDE)
- MariaDB Server (DB)
- Tomcat 9 (Web Server)
- FileZilla (Deploy della Web App)

Struttura della Web App (MVC):

- Back End (Model):

Modello ER del DB

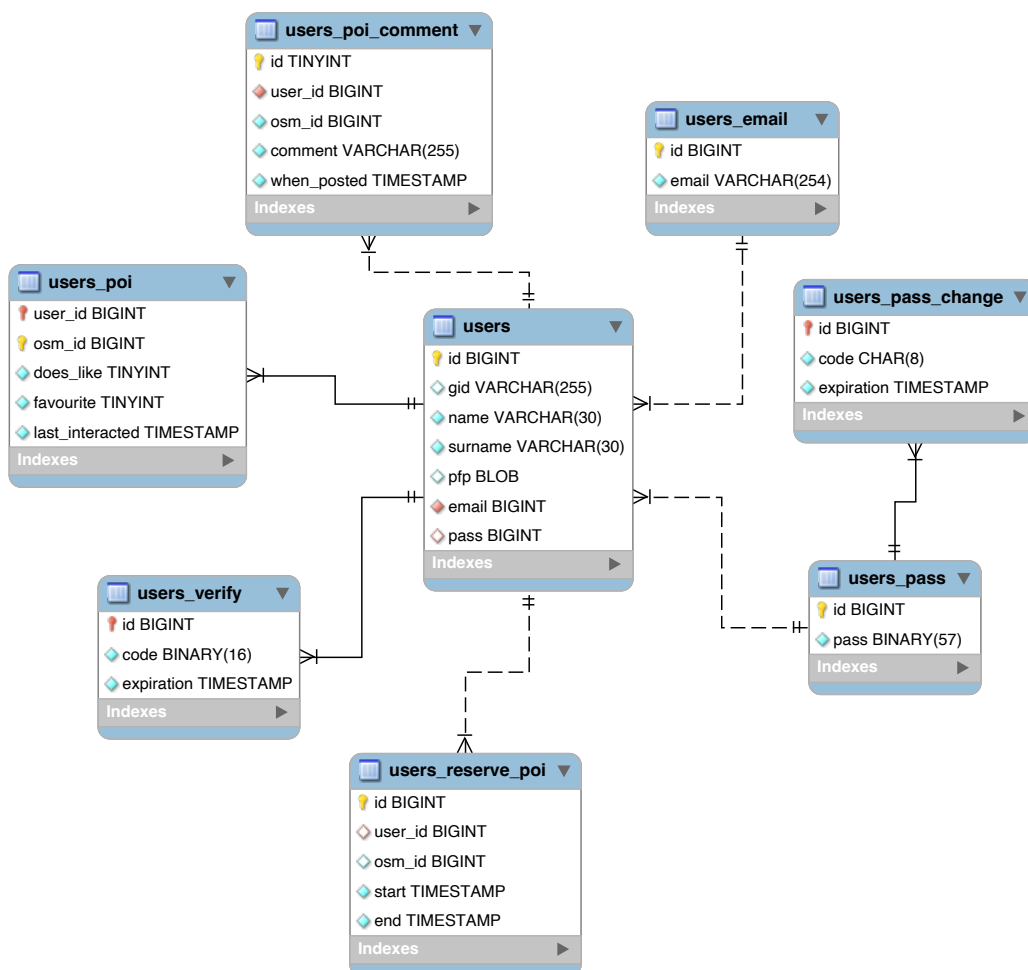


Figura 1 - Generato tramite MySQLWorkbench

Nel DB sono state inoltre implementate:

- Funzioni, per permettere un'esecuzione rapida, ottimizzata e soprattutto controllata da parte dei controller che dialogano con il DBMS.
- Trigger per garantire la validità degli inserimenti.
- Eventi per "pulire" le tabelle con dati scaduti.

Entità:

1. users:

Contiene il nome, cognome, foto profilo ed eventuale Google ID (gid) dell'utente, la password ed email si trovano in tabelle separate (tecnica della pseudonimizzazione per conformarsi al GDPR).

2. users_email e users_pass:

Contengono la vera e propria email e password (non in chiaro) dell'utente, la relazione tra l'utente e la password è opzionale in quanto sarebbe inutile nel caso di un utente che esegue l'accesso tramite Google.

3. users_verify e users_pass_change:

Entrambe le tabelle vengono utilizzate per immagazzinare dei codici in scadenza, generati quando l'utente crea un profilo nel modo "classico" o richiede il cambio della password.

4. users_poi e users_poi_comment:

Lo scopo di queste tabelle è quello di conservare le relazioni esistenti tra gli utenti e i luoghi, la prima tabella immagazzina le interazioni come i like e il salvataggio dei luoghi, e quando sono avvenute. La seconda registra i commenti degli utenti, le due tabelle sono state separate in modo da permettere all'utente di scrivere più di un commento.

5. users_reserve_poi:

Contiene le prenotazioni effettuate dagli utenti, può essere effettuata una prenotazione per luogo, e alla fine del periodo prenotato sarà possibile effettuarne un'altra. Questo stato è garantito grazie a un trigger.

- **Middleware (Controller)**

Il middleware, ovvero il componente che si occupa di mediare la comunicazione tra Front End e Back End, è composto da un insieme di controller (Servlet) implementate grazie alla libreria Java EE 8, lo scopo di queste Servlet è quello di elaborare e rispondere alle richieste provenienti dai client.

Il metodo di comunicazione applicato dalle Servlet è quello di esporre degli URI nella propria Web App alla quale è possibile inviare richieste HTTP, tutte le Servlet implementano delle logiche di *Session Filtering* in modo da evitare di far ripetere procedure ad un utente autenticato, o di elaborare richieste da parte di utenti non autorizzati.

URI esposti:

1. */signin*

Utilizzata per la registrazione di utenti “normali” (non Google).

Metodi

- a. POST: metodo utilizzato per inizializzare una nuova procedura di registrazione di un utente, esso non è ancora abilitato ad accedere al proprio profilo, in quanto deve prima verificare la propria email. Questa Servlet si occupa quindi di inviare al DB le informazioni necessarie alla creazione di un account, e di inviare all'utente l'email di verifica.
- b. GET: metodo utilizzato per completare la procedura di registrazione, prende come parametro un solo valore, ovvero il codice generato dalla richiesta POST (UUID) e richiede al DB la cancellazione del record con il codice appena inviato, in questo modo si evita che l'evento di “pulizia” cancelli il record assieme all'account nel caso in cui il codice risulti scaduto.

2. */login*

Utilizzata per l'accesso degli utenti al proprio account.

Metodi

- a. POST: metodo utilizzato per l'autenticazione di un utente e la creazione di una sessione, in seguito si viene rimandati alla Servlet */load* che si occupa del caricamento della “pagina principale” della Web App (mappa)

3. */google*

Utilizzata per gestire gli accessi con Google, richiamata automaticamente dall'apposito bottone in caso di accesso valido.

Metodi

- a. POST: metodo utilizzato per regolare gli accessi con Google, a livello di Servlet non vi è una distinzione tra accesso e registrazione come per gli utenti normali, entrambi vengono gestiti da questa Servlet che si occupa di validare il JWT ricevuto dall'API di Google, e in seguito di capire se si tratta di un account già esistente o se necessita di essere creato. Viene anche saltata la procedura di verifica, in quanto la email viene già "garantita" da Google. In seguito questa Servlet rimanda a */load*.

4. */load*

Utilizzata per il caricamento delle informazioni necessarie.

Metodi

- a. GET: metodo utilizzato per elaborare e inviare alla View i dati dell'utente richiedente, non vi è alcun parametro, il riconoscimento dell'utente avviene tramite la sessione creata in precedenza da */login* o */google*.

5. */image*

Utilizzata per la gestione delle foto profilo.

Metodi

- a. GET: metodo utilizzato per l'invio della foto profilo di un utente, per utilizzarla si ha bisogno di una sessione convalidata, prende come parametro l'id dell'utente della quale si vuole ricevere la foto profilo, in caso non vi sia il parametro id si riceve la propria. Questo metodo ha un funzionamento simile a quello di una REST API in quanto il vero e proprio rendering dell'immagine viene fatto lato client. Questa Servlet è stata implementata per l'evitare l'*embedding* dell'immagine all'interno del **src** del tag `img` HTML.

6. */logout*

Utilizzata per scollegarsi.

Metodi

- a. GET: metodo utilizzato per invalidare la sessione del richiedente, in questo modo il Web Server può rilasciare tutte le risorse legate ad essa.

7. */change*

Utilizzata per il cambio di password di un utente. Il suo funzionamento è molto simile a quello della Servlet che si occupa della registrazione.

Metodi

- a. POST: metodo utilizzato per inizializzare la procedura di cambio password. Si occupa di salvare il codice nel DB e di inviarlo via email all'utente.
- b. GET: metodo per finalizzare la procedura di cambio password. Prende come parametro il codice, la nuova password e la email del richiedente.

8. */getmeta*

Utilizzata dalla View per il rendering di un POI.

Metodi

- a. GET: metodo utilizzato per l'invio dei metadati (numero di like, commenti, e relazioni tra utente e POI) ad una View che in seguito si occuperà di elaborarli. Essa è una REST API in quanto si occupa semplicemente di richiedere dei dati al DB e di inviarli in forma di JSON al client. Prende come parametro l'id del POI e come id utente viene utilizzato quello legato alla sessione.

9. */updatemeta*

Utilizzata per creare o modificare le relazioni tra utenti e POI.

Metodi

- a. POST: metodo utilizzato per salvare i commenti lasciati dagli utenti nel DB, in seguito invia una risposta al client sull'esito dell'operazione.
- b. GET: metodo utilizzato per modificare o creare le relazioni (like e/o salvataggi) tra gli utenti e i POI.
- c. DELETE: metodo utilizzato per eliminare un commento, prende come parametro l'id del commento da eliminare.

10. */reserve*

Utilizzata per creare o eliminare delle prenotazioni.

Metodi

- a. POST: metodo utilizzato per inserire una prenotazione da parte di un utente nel DB, in seguito rimanda ad una pagina che mostra il risultato dell'operazione ed invia un'email all'utente con un resoconto di essa.
- b. GET: metodo utilizzato per cancellare una prenotazione, essa prende come parametro l'id della prenotazione che si desidera cancellare.

11. /changeprofile

Utilizzata per modificare le informazioni di utenti già esistenti.

Metodi

- a. GET: metodo utilizzato per elaborare i dati necessari alla pagina di modifica profilo utente. In seguito rimanda alla pagina elaborata.
- b. POST: metodo utilizzato per aggiornare i dati relativi ad un utente, prende come parametro i dati che si desidera modificare, in caso non ne venga fornito alcuno la richiesta non ha alcun effetto.

• Front End (View)

Il Front End è stato implementato tramite l'utilizzo delle JSP (JavaServer Pages) e di bundle JavaScript compilati tramite Webpack, le JSP (View) hanno lo scopo di mostrare i dati elaborati dai vari controller, mentre gli script hanno il compito di gestire il funzionamento della mappa e di effettuare vari controlli sui dati inseriti.

La mappa è stata implementata tramite OpenLayers, una libreria open source scritta in JavaScript che permette l'implementazione in modo abbastanza intuitivo di mappe interattive. I dati utilizzati per elaborare la mappa ed i relativi POI sono provenienti da OpenStreetMap, un progetto, anch'esso open source, che fornisce l'accesso gratuito ai propri dati e vari servizi utili per interagire con essi.

Tra i vari servizi che OpenStreetMap offre ci sono i seguenti:

- **Nominatim**: esso contiene dati molto precisi su vari POI nel mondo, nel progetto vengono effettuate vari tipi di richieste all'API di Nominatim.
Viene utilizzato:
 1. /reverse per ricavare le informazioni su un POI partendo dalle sue coordinate.
 2. /search per l'implementazione di una barra di ricerca.
 3. /lookup per il caricamento dei dati relativi ai posti salvati dall'utente.
- **Overpass**: esso è un servizio alla quale vengono effettuate richieste strutturate, basato su un linguaggio di **Querying** definito dai suoi creatori, nel progetto viene utilizzato per richiedere la posizione dei vari POI presenti in una determinata **Bounding Box**, essa è rappresentata dallo schermo del Client più un buffer che varia in base allo zoom applicato sulla mappa.

Per l'elaborazione di un POI vengono anche effettuate delle query alle Servlet del progetto, esse contengono vari metadati legati ai POI, tali metadati sono: il numero di like, se l'utente ha messo like al POI, se l'utente ha salvato il POI, se l'utente ha una prenotazione "attiva" al POI, i commenti del POI e i commenti lasciati dall'utente al POI.

Durante l'esecuzione della Web App si cerca di conservare il più possibile i dati legati ad un POI, cercando soprattutto di limitare le richieste ai servizi esterni di **OSM** (OpenStreetMap), questi meccanismi di *caching* si basano sulla conservazione delle query già effettuate in passato, essendo che difficilmente i dati geografici legati ad un POI cambiano.

La principale criticità è che ciò può compromettere l'aggiornamento dei metadati in tempo reale, ad esempio se un altro utente mette like al POI esso **non sarà visibile immediatamente** per gli altri utenti, in caso abbiano il POI nella cache.

Un altro meccanismo di *ottimizzazione* adottato è quello di limitare le query di aggiornamento della relazione tra POI e utente, quando l'utente mette like o salva un POI l'azione non viene immediatamente comunicata al Server.

La relazione viene aggiornata solamente in locale dando un'**illusione** di responsività istantanea all'utente, in realtà la modifica di tale relazione viene comunicata solo dopo che il popup del POI viene chiuso.